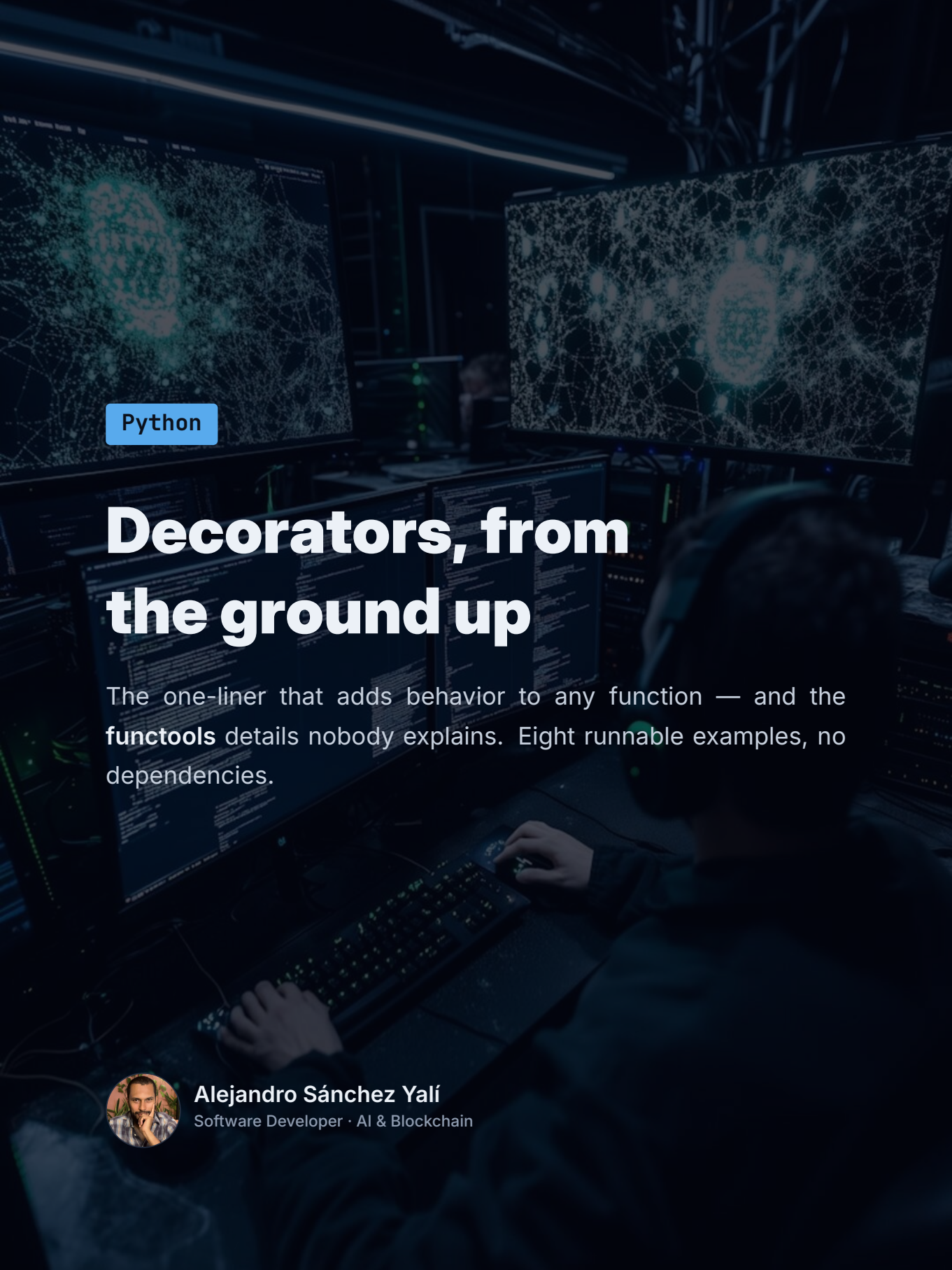

Python

Decorators, from the ground up

The one-liner that adds behavior to any function — and the **functools** details nobody explains. Eight runnable examples, no dependencies.

Alejandro Sánchez Yalí
Software Developer · AI & Blockchain

PYTHON | DECORATORS

1 What a decorator really is

A decorator takes a function and returns a new one that runs extra logic around it — logging, timing, caching, access checks — without changing the original code. That is the whole idea. The `@` syntax is sugar: writing `@log` above `greet` is exactly `greet = log(greet)`.

pattern.py

```python
def log(fn):
    def wrapper(*args, **kwargs):
        print(f"-> {fn.__name__}")
        return fn(*args, **kwargs)
    return wrapper

@log
def greet(name):
    return f"Hi, {name}"
```

`log` receives a function and hands back `wrapper`; Python then rebinds `greet` to that wrapper. The `*args, **kwargs` carry the whole call through untouched, so the same decorator fits a function with two positional arguments or one with keyword-only arguments.

> WHY IT MATTERS
>
> This is how frameworks add behavior declaratively. Flask's `@app.route`, `@property`, `@dataclass` — all the same mechanism. One line above a function instead of ten inside it.

2 functools.wraps, the one you keep forgetting

Wrap a function naively and you overwrite its identity. `add.__name__` becomes `"wrapper"`, the docstring turns into `None`, and every tool that reads them — your debugger, `help()`, pytest's output — now reports the wrapper instead

of your function.

wraps.py

```python
from functools import wraps

def log(fn):
    @wraps(fn)          # copy __name__, __doc__, __wrapped__, ...
    def wrapper(*a, **k):
        return fn(*a, **k)
    return wrapper
```

`@wraps(fn)` copies the metadata from the original onto the wrapper. Run `02_wraps.py` and the difference is plain:

> **GOTCHA**
>
> Add `@wraps` to **every** decorator you write. It costs one line; skipping it costs an afternoon of debugging a stack trace that points at `wrapper` in three different places.

3 Decorators that take arguments

A decorator with its own arguments needs one more layer of nesting: the **argument**, then the **function**, then the **wrapper**.

args.py

```python
def retry(times):
    def deco(fn):
        def wrapper(*a, **k):
            ...            # sees both `times` and `fn`
        return wrapper
    return deco

@retry(3)
def fetch(): ...
```

Read `@retry(3)` as two steps. First `retry(3)` runs and returns `deco`; then `deco` decorates `fetch`. Forget to return a layer, or try to take `times` and `fn` in the same function, and it fails with a confusing `TypeError`. A complete `retry` with backoff lives in `03_decorator_with_args.py`.

4 Stacking order

Stacked decorators apply **bottom-up**: the one nearest the function wraps first.

order.py

```python
@bold
@italic
def greet(): ...
# same as: greet = bold(italic(greet))
```

So `italic` runs closest to `greet`, and `bold` wraps the result. This is not cosmetic. Put a caching decorator above an auth check and you might serve a cached response to an unauthorized caller; swap them and you fix it. When two decorators interact, the order is part of the logic.

PYTHON | DECORATORS

5 Don't reinvent it: the stdlib ships decorators

Before writing a caching or dispatch decorator, look in `functools`. `lru_cache` memoizes a function by its arguments, turning exponential `fib` into linear.

cache.py

```python
from functools import lru_cache

@lru_cache(maxsize=None)
def fib(n):
    return n if n < 2 else fib(n-1) + fib(n-2)
```

`fib(35)` drops from millions of calls to 36, and `fib.cache_info()` reports `hits=33, misses=36`. A hand-written timing decorator (`codes/04_timer.py`) is another good one to keep around.

> PRO TIP
>
> Learn what `functools` already gives you: `lru_cache`, `cached_property` (compute once per instance), `singledispatch` (overload by type), `partial`, and `wraps` itself.

6 Decorating classes

@property — methods that read like data

`@property` turns a method into a read-only attribute: you read `c.area`, not `c.area()`. Pair it with a setter to guard writes.

property.py

```python
class Circle:
    def __init__(self, radius):
        self.radius = radius        # goes through the setter
    @property
    def radius(self):
        return self._radius
    @radius.setter
    def radius(self, value):
        if value ⩽ 0:
            raise ValueError("radius must be positive")
        self._radius = value
    @property
    def area(self):
        return math.pi * self._radius ** 2
```

Now `c.radius = -1` raises instead of silently corrupting the object, and `area` is always computed from a valid radius.

classmethod vs staticmethod

`@classmethod` receives the class as `cls`; `@staticmethod` receives nothing.

methods.py

```python
class User:
    @classmethod
    def from_json(cls, data):       # alternate constructor
        return cls(data["name"], data["email"])
    @staticmethod
    def valid(email):               # helper, doesn't touch the
    class
        return "@" in email
```

Use `classmethod` for alternate constructors — because it gets `cls`, `User.from_json(...)` keeps working in subclasses. Use `staticmethod` for helpers that belong to the class conceptually but need neither the instance nor the class.

7 When you need state: class-based decorators

Three nested functions get hard to read once the decorator has to remember something — a counter, a rate limit, a config. A class with `__call__` is clearer, and the state lives on the instance.

class-based.py

```
class CountCalls:
    def __init__(self, fn):
        self.fn = fn
        self.count = 0
        update_wrapper(self, fn)    # the class-based @wraps
    def __call__(self, *a, **k):
        self.count += 1
        return self.fn(*a, **k)
```

`update_wrapper(self, fn)` is the class-based equivalent of `@wraps`. After `@CountCalls`, `ping.count` is real, inspectable state.

8 Common mistakes

- **No `@wraps`.** Broken `__name__`, lost docstrings, confused tooling. Ten seconds to fix; hours to debug.
- **Confusing definition time with call time.** The decorator body runs once, when the function is defined. The `wrapper` runs on every call. Put expensive setup in the decorator, not the wrapper.

- **Shared mutable state in the closure.** A `list` or `dict` created in the decorator is shared across every call — a feature for a cache, a bug for everything else.
- **Nested functions when you need state.** If it holds state or config, use a class with `__call__`.

9 Read more

Every snippet above ships as a complete, dependency-free file in `codes/`. Two references worth bookmarking:

- Python docs — functools: docs.python.org/3/library/functools
- PEP 318 — the decorator syntax: peps.python.org/pep-0318

Read the full guide

Eight runnable, dependency-free examples plus the complete write-up — clone the repo or read it right on GitHub.

Alejandro Sánchez Yalí
Software Developer · AI & Blockchain